

# DUNGEON M-AI-ND: ANFORDERUNGEN

## FEATURES



As Projekt soll um die folgenden Features ergänzt werden:

### 1. Interaktive Timeline:

Im Backend werden die transkribierten Daten analysiert, um signifikante Ereignisse zu identifizieren. Diese werden mit Zeitstempeln und Kurztiteln versehen und im Frontend auf einem interaktiven Zeitstrahl angezeigt.

### 2. Geografische Kartierung:

Das System extrahiert Ortsnamen und deren räumliche Zusammenhänge aus den Aufzeichnungen. Basierend auf diesen Relationen wird eine geografische Karte generiert. Durch die Verknüpfung mit der Timeline lässt sich die Reise eurer Gruppe punktgenau nachvollziehen.

### 3. Erweitertes Charaktersheet:

Die aktuelle, rudimentäre Anzeige der Lebenspunkte im Frontend wird zu einem vollwertigen Charaktermanagementsystem ausgebaut, das alle relevanten Attributwerte sowie ein dynamisches Inventar umfasst.

einzelne Funktion (User Story) sollte den Umfänge einer einzelnen Nutzerinteraktion abdecken (Ausgangszustand -> Nutzer drückt Knopf -> Endzustand) und eine Sprintdauer nicht übersteigen.<sup>1</sup>

#### BEISPIEL USERSTORY:

Initial State: The user is on the web page and sees the "Start Recording" button. Their browser has not yet granted microphone access to the website.

User Action: The user clicks on the "Start Recording" button on the web page.

End State: The browser prompts the user to grant or deny microphone permission for the website. If granted, the recording functionality becomes active, allowing the user to capture audio. If denied, a clear message is displayed informing the user that microphone access is required for the recording feature to work.

Innerhalb eines Releases finden wöchentlich Sprintmeetings statt.

Zum Enddatum eines Releases sollen alle geplanten Features implementiert und als lauffähige Anwendung (Startskript/Docker) auf dem Mainbranch verfügbar sein.

Die Präsentation der Releaseergebnisse erfolgt an einem vereinbarten Termin in den folgenden 2 Wochen.

## ORGANISATION



As Projekt wird in 6 Releases unterteilt.

- Release 1: 14.6.2026  
In existierenden Code einarbeiten und Prototyp für die Extraktion von zeitlichen und örtlichen Entitäten aus den Embeddings erstellen.
- Release 2: 31.7.2026 Interaktive Timeline implementieren.
- Release 3: 30.9.2026 Die Geografische Kartierung implementieren 1.
- Release 4: 30.11.2027 Die Geografische Kartierung implementieren 2.
- Release 5: 31.1.2027 Das Erweiterte Charaktersheet implementieren.
- Release 6: 31.3.2027 Projekt und Abgabe finalisieren.

## RELEASESTRUKTUR

Ein jedes Release beginnt mit dem definieren der zu implementierenden Features. Dazu sollen (ausser Release 1 und 6) zunächst Mockups der neuen Features und UI erstellt werden. Anhand derer dann eine Liste der einzelnen Funktionen erstellt wird, die benötigt werden um die Features umzusetzen. Eine

## SPRINTMEETINGS

Der Sinn der Sprintmeetings ist es, dass der Austausch zwischen den Projektteilnehmern gefördert wird, um ein reibungsloses Zusammenspiel der einzelnen Projektkomponenten zu erzielen. AuSSerdem begünstigen sie die kontinuierliche Arbeit am Projekt um Crunch zum Ende der Releases und des Projektes zu minimieren. Daher soll sich in einem Sprintmeeting über den aktuellen Arbeitsstand ausgetauscht werden, welche Funktionen implementiert wurden, welche Aufgaben länger dauern als erwartet, welche Probleme aufgetreten sind und welche Aufgaben als nächstes anstehen und wer davon welche übernehmen kann. Für jedes dieser Meetings soll eine kruze Doku erstellt werden, wie unter Abgabe/Sprintmeetingdokumentation.

## DOKUMENTATION



Ährend des Projektes sollen mehrere Dokumentationsartefakte erstellt werden. Diese haben zum Teil mehrere Zwecke. Zum einen wird damit eine geleistete Arbeit festhalten und dokumentiert. Zum anderen erleichtert es

<sup>1</sup>Es wird empfohlen die Userstories/Funktionen, aus der Textdatei in eine Software wie Jira, Youtrack oder Planka zu überführen um besser nachverfolgen zu können, wer was wann erledigt (hat) und was noch fehlt.

anderen Personen eure Arbeit nachzuvollziehen und evtl. zu erweitern.

## README

Im Rootverzeichnis des Repositories muss eine Readme liegen, die kurz den Zweck der Software, die Funktionen sowie die Installation und das Starten der Software erklärt.

## NUTZERHANDBUCH

Eine kurze PDF-Datei, die anhand von Screenshots, einem Nutzer die Funktionen der Software zeigt und erklärt, wie diese zu nutzen sind.

## SPRINTMEETINGDOKUMENTATION

Eine Datei mit dem Kurzdokus zu den wöchentlichen Sprintmeetings. Für jedes Meeting sollte enthalten sein, wer teilgenommen hat, welche Aufgaben abgeschlossen wurden, welche Probleme aufgetreten sind, und was als nächstes ansteht.

### BEISPIEL:

Student 1 didnt finish the fix of the rejoin bug and will do this till next week. Also he will put the configview and rulebookview as modal on the homeview.

Student 2 refactored the homeview and split it in logic parts to increase usability, and replaced the "kick"-button. Till next week she will read the rules if, when a player gains max-hp the actual hp are increased to or stay. Also she will work on the login-view to make it look similar to the home-view.

Student 3 reworked the player-site as modal on the homeview. Till next week he will rework the Player overview, that first the player needs to record a audio snippet and after that the player can see his health and abilities.

Student 4 will work on the logic to finish the transcription, that the user will see if the backend still transcribes the audio session - maybe luca will pick this up, to set it as a flag to save a session.

Student 5 reworked the save/load-logic in a way, that its now possible to save or play in campaigns where a session can be saved under a certain campaign. In a single campaign can be multiple sessions saved. Till next week he will work further on the campaign-logic and create a block to save a session while the backend still transcribes the audio.

Student 6 couldnt join today.

## CODE DOKUMENTATION

Für die Dokumentation ist die typische, sprachabhängige, In-Code-Doku ausreichend.

## APIDOKUMENTATION

Wenn ihr in einem Release neue Funktionen zu einer API hinzugefügt habt, oder eine neue API erstellt habe, sollt ihr ein kurzes Dokument erstellen, in dem die neue Funktion und ihr Nutzen beschrieben wird.

## ARBEITSDOKUMENTATION

Welchen Beitrag ihr zu dem Projekt geleistet habt wird über eure git-Commits dokumentiert.

Daher solltet ihr regelmässig Commits erstellen und auf Github pushen. Da das Projekt mit 12 Credits bewertet ist, entspricht das etwa einer theoretischen Wochenzeit von 7,5 Stunden. Dementsprechend bilden 5 Commits pro Person pro Woche/Sprint eine gute Grundlage.

## ABGABE



Inhalt der Abgabe ist:

### 1. Die Projekt Software:

Alle Features müssen auf Github in den main Branch gemerged worden sein. Die Software muss über ein Skript oder einen kurzen Dockerbefehl gestartet werden können und darf keine Bugs enthalten.

### 2. Readme

### 3. Nutzerhandbuch

### 4. Sprintmeetingdokumentation

### 5. Demovideo:

Ein 3-5 Minütiges Video ( ≤ 200MB) in dem ihr das Ergebnis eures Projektes demonstriert.

### 6. Kurze Releasedokumentation:

Ein Zusammenschnitt der für jeden Release erstellten Dokumente. Geplante Features, Mockups, Fertiggestellte Features, API-Doku, ...

# DUNGEON M-AI-ND: REQUIREMENTS

## FEATURES



The project is to be expanded with the following features:

### 1. Interactive Timeline:

The transcribed data will be analyzed in the backend to identify significant events. These will be assigned timestamps and short titles and displayed on an interactive timeline in the frontend.

### 2. Geographic Mapping:

The system extracts place names and their spatial relationships from the recordings. A geographic map is generated based on these relations. By linking this with the timeline, your group's journey can be tracked with pinpoint accuracy.

### 3. Advanced Character Sheet:

The current, rudimentary display of hit points in the frontend will be expanded into a full-fledged character management system, encompassing all relevant attribute values and a dynamic inventory.

## ORGANIZATION



The project is divided into 6 releases.

- Release 1: June 14, 2026  
Familiarization with existing code and creation of a prototype for extracting temporal and local entities from the embeddings.
- Release 2: July 31, 2026  
Implement the interactive timeline.
- Release 3: September 30, 2026  
Implement Geographic Mapping (Part 1).
- Release 4: November 30, 2026  
Implement Geographic Mapping (Part 2).
- Release 5: January 31, 2027  
Implement the Advanced Character Sheet.
- Release 6: March 31, 2027  
Finalize project and submission.

## RELEASE STRUCTURE

Each release begins with defining the features to be implemented. For this purpose (except for Release 1 and 6), mockups of the new features and UI should be created first. Based on these, a list of individual functions required to implement the features will be generated. A single function (User Story) should cover the scope of a single user interaction (Initial State -> User presses button -> End State) and should

not exceed the duration of one sprint.<sup>2</sup>

### EXAMPLE USER STORY:

Initial State: The user is on the web page and sees the "Start Recording" button. Their browser has not yet granted microphone access to the website.

User Action: The user clicks on the "Start Recording" button on the web page.

End State: The browser prompts the user to grant or deny microphone permission for the website. If granted, the recording functionality becomes active, allowing the user to capture audio. If denied, a clear message is displayed informing the user that microphone access is required for the recording feature to work.

Weekly sprint meetings take place within a release.

By the end date of a release, all planned features must be implemented and available on the main branch as a runnable application (startup script/Docker).

The presentation of the release results will take place at an agreed-upon date within the following 2 weeks.

## SPRINT MEETINGS

The purpose of the sprint meetings is to promote communication between project participants to ensure smooth interaction between the individual project components. Furthermore, they encourage continuous work on the project to minimize "crunch" at the end of the releases and the project. Therefore, the current state of work should be discussed in a sprint meeting: which functions have been implemented, which tasks are taking longer than expected, which problems have occurred, which tasks are up next, and who can take them over. Brief documentation must be created for each of these meetings, as described under Submission/Sprint Meeting Documentation.

## DOCUMENTATION



Several documentation artifacts are to be created during the project. These serve multiple purposes: on the one hand, they record and document the work you have performed; on the other, they make it easier for others to understand and potentially expand upon your work.

## README

A Readme file must be located in the root directory of the repository, briefly explaining the

<sup>2</sup>It is recommended to transfer the user stories/functions from the text file into software such as Jira, Youtrack, or Planka to better track who finished what and when, and what is still missing.

purpose of the software, its functions, as well as installation and startup instructions.

## USER MANUAL

A short PDF file that uses screenshots to show and explain the software's functions to a user and how to use them.

## SPRINT MEETING DOCUMENTATION

A file containing the brief documentation of the weekly sprint meetings. For each meeting, it should include who participated, which tasks were completed, what problems occurred, and what is scheduled next.

### EXAMPLE:

Student 1 didn't finish the fix of the rejoin bug and will do this till next week. Also, he will put the configview and rulebookview as a modal on the homeview.

Student 2 refactored the homeview and split it into logical parts to increase usability, and replaced the "kick" button. Till next week she will read the rules to see if, when a player gains max-HP, the actual HP are increased as well or stay the same. Also, she will work on the login-view to make it look similar to the home-view.

Student 3 reworked the player-site as a modal on the homeview. Till next week he will rework the Player overview; first, the player needs to record an audio snippet, and after that, the player can see his health and abilities.

Student 4 will work on the logic to finish the transcription, so that the user will see if the backend is still transcribing the audio session - maybe Luca will pick this up to set it as a flag to save a session.

Student 5 reworked the save/load logic so that it is now possible to save or play in campaigns where a session can be saved under a certain campaign. Multiple sessions can be saved within a single campaign. Till next week he will continue working on the campaign logic and create a block to save a session while the backend is still transcribing the audio.

Student 6 couldn't join today.

## CODE DOCUMENTATION

Typical, language-dependent, in-code documentation is sufficient.

## API DOCUMENTATION

If you have added new functions to an API or created a new API during a release, you should create a short document describing the new function and its utility.

## WORK DOCUMENTATION

Your contribution to the project is documented via your git commits. Therefore, you should create commits regularly and push them to GitHub. Since the project is valued at 12 credits, this corresponds to a theoretical weekly time of approximately 7.5 hours. Accordingly, 5 commits per person per week/sprint constitute a good baseline.

# SUBMISSION



He submission consists of:

### 1. The Project Software:

All features must have been merged into the main branch on GitHub. The software must be startable via a script or a short Docker command and must not contain any bugs.

### 2. Readme

### 3. User Manual

### 4. Sprint Meeting Documentation

### 5. Demo Video:

A 3-5 minute video ( $\leq 200\text{MB}$ ) demonstrating the results of your project.

### 6. Short Release Documentation:

A compilation of the documents created for each release: planned features, mockups, completed features, API documentation, etc.